A Tiered Compilation Architecture for Low-Latency Wasm Runtimes in Edge Computing

Master's Thesis Defense

Hsiang-Yu Lee (李祥宇)
Advisor: Prof. Shih-Wei Liao

May 22, 2026

Department of Computer Science and Information Engineering
National Taiwan University

Outline

Motivation

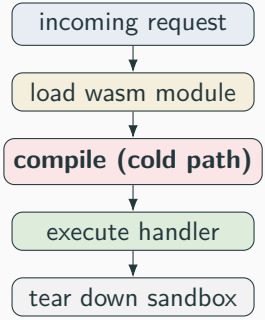
Edge computing reshapes the workload

Edge nodes face two tight budgets.

- **Response time:** tens of milliseconds per request.
- **Resources:** 1–2 GB RAM and few vCPUs.

The workload mix is short-lived, stateless functions that start and exit constantly.

WebAssembly fits the role. The format is compact, language-agnostic, sandboxed, and predictable. Fastly Compute, Cloudflare Workers, and Fermyon Spin all run WebAssembly in production.



Per-request lifecycle.

The WasmEdge cold-start bottleneck

WasmEdge is the leading server-side Wasm runtime. Microsoft, ByteDance, and Sony all run WasmEdge in production.

WasmEdge compiles with LLVM end-to-end. The LLVM pipeline produces near-native code. The pipeline is also expensive:

- Compile time grows faster than module size.
- Memory usage during compilation is large.
- Every cold start pays the full compile cost.

The structural problem. A single-tier compiler treats every invocation the same way. A request that runs for one microsecond pays the same compile cost as one that runs for one minute.

Cold-start latency

Definition

Time from request arrival to the first machine-code instruction of the requested wasm function. Assumes no ready-to-run compiled instance of the module is resident.

Inside the window

- load, parse, validate
- compile
- instantiate

Outside the window

- the wasm function's own execution
- reported as steady-state work

Execution time is highly skewed

A small fraction of functions accounts for the bulk of run-time.

- Heavy optimization on cold functions wastes the compile budget.
- Cold functions receive optimization they do not need.
- Hot functions wait behind cold compilation before they can start.

The established solution is tiered compilation.

- A **Tier-1** baseline compiles every function fast.
- A **Tier-2** optimizer recompiles hot functions later, guided by profile data.
- V8, JavaScriptCore, and SpiderMonkey all use this pattern.

WasmEdge has no baseline tier, no profile-driven promotion, and no mid-execution code replacement.

Related work

Browser-engine Wasm pipelines. V8 pairs Liftoff and TurboFan. JavaScriptCore runs IPInt, BBQ, and OMG. SpiderMonkey runs Baseline and Ion. All three use function-entry promotion. Public documentation reports no OSR between the baseline JIT and the optimizing JIT.

Server-side Wasm pipelines. Wasmtime offers Cranelift (optimizing) and Winch (baseline). The runtime does not currently tier up between them.

OSR in language VMs. HotSpot has supported OSR for over two decades. .NET 7 added OSR in 2022 with the one-shot-caller motivation this thesis also targets. Tracing JITs (PyPy, LuaJIT) sidestep OSR through trace recording.

This thesis fills a gap. No published server-side WebAssembly runtime combines a non-LLVM Sea-of-Nodes baseline, selective LLVM-frontend reuse, and loop-level OSR.

Problem Statement and Contributions

Three sub-problems

Goal. Cut cold-start latency without giving up the steady-state throughput that LLVM delivers on hot code.

Tier-1 baseline. Compile every wasm function fast at module load. Cover the full WebAssembly instruction set the workloads use. Reuse the existing WasmEdge runtime contracts.

Tier-2 optimizer. Reuse the LLVM frontend to recompile hot functions. Bridge the two tiers' calling conventions. Hide the LLVM compile cost from the user thread.

In-flight migration. A function called once spends its run-time inside one long loop. The prologue never re-fires, so entry-table promotion delivers no speedup. A separate mechanism must migrate the running frame mid-loop.

Contributions

1. **Tier-1 baseline JIT for WasmEdge.**
   Integrate a lightweight Sea-of-Nodes library.
   Translate wasm bytecode to sea-of-nodes IR nodes.
   Embed per-function and per-loop profile counters directly in compiled code.

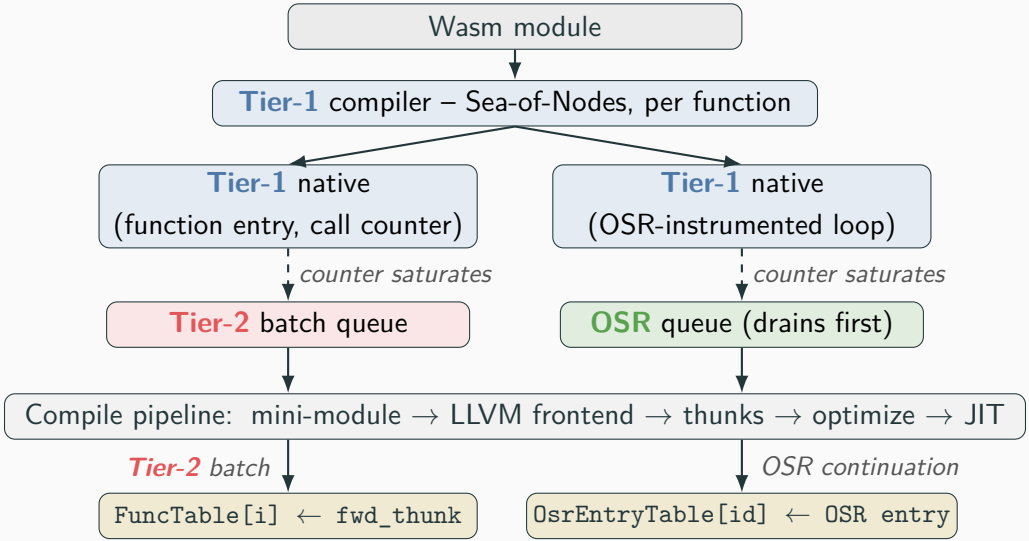
2. **Tier-2 selective LLVM-frontend recompilation.**
   Design a heuristic to synthesizes a mini-module per batch.
   Design three ABI bridge thunks to bridge the calling convention gap between the 2 tiers.

3. **On-Stack Replacement.**
   Instrument every outermost loop with a three-state back-edge diamond.
   Synthesize a continuation that starts execution at the loop header.

4. **Empirical evaluation.**
   Report and discuss a step-by-step experiment on 39 strengthened sightglass kernels.

Architecture Overview

System architecture



Pipeline at a glance

- The user thread runs **Tier-1** native code and trips the embedded counters.
- A heuristic selects a batch of functions to promote to **Tier-2** and synthesizes a mini-module for the batch.
- Two promotion paths feed two queues: function-entry promotion and loop OSR.
- A worker thread compiles **Tier-2** batches and OSR continuations in the background.
- The worker publishes each result with one atomic store into the dispatch slot.

Tier-1: Baseline JIT

Tier-1: a lightweight Sea-of-Nodes baseline

Code generator: dstogov/ir. A small Sea-of-Nodes JIT library, descended from the Zend JIT in PHP 8.

- The library runs a fixed, linear pipeline. No pass manager.
- Compile latency stays bounded and predictable.
- The codebase is small enough to vendor and modify.

Advantages of sea-of-nodes IR.

- **SSA by construction.** Values are graph nodes. Definitions and uses are explicit edges. No separate SSA-construction phase.
- **One graph for control flow and data.** A single traversal covers both. Passes do not need parallel analyses.
- **Cheap global code motion.** Floating nodes let the scheduler choose placement after optimization.

Tier-1: from wasm to native

Each wasm function is compiled at module instantiation. The pipeline runs one linear pass through five stages.

1. **Walk the wasm body in source order.** For each instruction, pop operand IR refs off a value stack, emit IR nodes, push the result back.
2. **Single-pass SSA.** Locals become SSA values. PHIs appear only at loop headers, only for locals the loop modifies.
3. **Insert profile instrumentation.** The prologue increments a per-function call counter. Outermost loops get the three-state OSR diamond on every back-edge.
4. **Run the `dstogov/ir` backend.** Folding, value numbering, scheduling, register allocation, code emission.
5. **Publish.** Store the native entry pointer in `FuncTable[FuncIdx]`.

Tier-2: Selective LLVM Recompilation

Batch composition: walk up, then bounded BFS

Promoting only the threshold-tripping function is not enough. The function that trips the counter is often a small leaf inside a larger hot caller. A leaf promoted alone leaves its caller in **Tier-1**, so the optimizer cannot inline across the boundary.

Algorithm 2

1. **Walk up one hop.** Pick the hottest direct caller as the batch root.
2. **Bounded BFS.** Traverse direct callees from the root up to a depth limit and a batch size limit.
3. **Two-test admission** for each candidate callee:
   - Dynamic test: the call counter exceeds zero.
   - Static test: the caller body references the callee at least twice.

The static test admits siblings whose dynamic counters lag behind but that the optimizer will still benefit from seeing.

Three ABI bridge thunks

Tier-1 passes arguments in a flat 64-bit buffer. LLVM passes arguments in typed registers. Three thunks bridge the mismatch, one per direction of cross-tier dispatch.

Thunk	Direction	Role
`fwd_thunk`	**Tier-1**→**Tier-2**	Lives in `FuncTable` after publication. Repacks **Tier-1** arguments into typed parameters and calls the **Tier-2** body.
`t1_thunk`	**Tier-2**→**Tier-1**	Replaces a non-batch stub body inside the mini-module. Repacks typed arguments back into the **Tier-1** buffer and dispatches under the **Tier-1** convention.
`entry_thunk`	LLVM→**Tier-1**	Sits in front of every **Tier-1** function. Catches indirect calls from **Tier-2**, OSR, or AOT code that target a **Tier-1** function.

Each thunk handles one direction. Neither tier needs to know the other tier's ABI.

Mini-module synthesis & Promotion

Algorithm 1

1. Deep-copy the original wasm module.
2. Replace every defined function outside the batch with a small stub.
3. Run the LLVM frontend on the mini-module unoptimized.
4. Post-process the stubs with thunks, and prune unused stubs.
5. Run the LLVM optimizer on the mini-module.
6. Emit native code and publish the `fwd_thunk` pointer into `FuncTable`.

We run the LLVM frontend unoptimized before post-processing to preserve the call sites that post-processing rewrites into thunk calls.

On-Stack Replacement

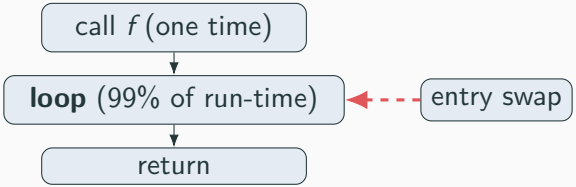
The one-shot-caller problem

A function called once, spending its run-time inside one long loop.

This is the shape of cryptographic kernels, parsing loops, and many serverless handlers.

Function-entry promotion delivers no speedup.

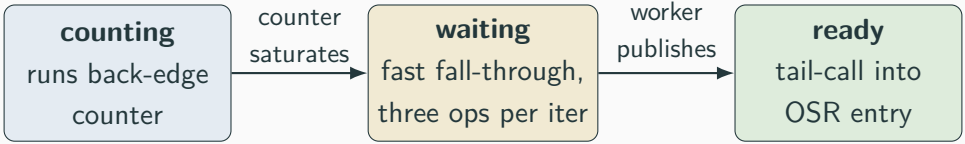
- Entry promotion redirects the next call to the function.
- This function is called once. There is no next call.
- The **Tier-1** frame keeps running the loop until return.



no future call to redirect

Three-state back-edge instrumentation

Each outermost loop reads one slot of `OsrEntryTable` on every back-edge. The slot encodes a three-state machine.



Steady-state cost. Three operations per iteration in the *waiting* state. An earlier two-array design needed six. Full pseudocode in the backup slides.

OSR continuation entry synthesis

The continuation reuses the mini-module mechanism with three changes.

1. **Parameter list.** The new function takes the original parameters and the original locals, flattened into one list. The caller passes the running frame's state in through the call.
2. **Function body.** The body opens the enclosing control constructs, then resumes the original instruction stream from the loop header.
3. **Fresh function slot.** The continuation lives in a new slot. The original function stays untouched. Calls inside the continuation dispatch back to the original through `t1_thunk`.

Two queues share one worker. The OSR queue is drained first.
Because an OSR continuation must publish before the running loop exits.

Evaluation

Methodology

Benchmark suite: sightglass-strong.
Upstream sightglass kernels run in well under a second. The strengthened suite extends each kernel's **Tier-1**-only run-time into a 5–10 s band. **Tier-2** now has enough post-switch runway to amortize the worker's compile cost.

39 analyzed kernels group into five execution shapes.

- **Frequent-call** (17): one hot function called many times.
- **Hot-leaf** (6): a small helper is called many times and becomes hot before the hot caller.
- **One-shot-caller** (7): outer function called once, dominated by one loop.
- **Resistant** (4): too little run-time or no inline opportunity.
- **Compile-dominated** (7): LLVM compile cost dwarfs the wasm work.

Three configurations, three steps

Each step turns on one mechanism on top of the previous step.

Step	t1/t2	LLVM/t2	Mechanism added
Step 1 – promote hot callees only	0.97×	0.76×	function-entry promotion
Step 2 – walk-up + bounded BFS	1.03×	0.80×	better batch composition
Step 3 – on-stack replacement (full system)	**1.11**×	**0.86**×	in-flight migration

- Geometric mean across all 39 kernels.
- t1/t2: how much faster the full system runs than **Tier-1** alone.
- LLVM/t2: how close the full system gets to whole-module LLVM JIT.
  A value of 0.86× means the full system recovers 86% of LLVM throughput.

Step 2: walk-up rescues hot-leaf kernels

Step 1 batched only the leaf that tripped the counter. The optimizer had nothing big enough to inline into. Step 2 rolls the surrounding caller into the batch.

Kernel	Step 1 LLVM/t2	Step 2 LLVM/t2	swing
hashset	0.62×	0.84×	+0.22
shootout-ed25519	0.38×	0.58×	+0.20
rust-json	0.62×	0.75×	+0.13
rust-compression	0.75×	0.87×	+0.12
regex	0.85×	0.95×	+0.10
pulldown-cmark	0.75×	0.84×	+0.09
gcc-loops	0.74×	0.83×	+0.09

Two groups still lag after Step 2. One-shot-caller kernels (shootout-ctype 0.37×, blind-sig 0.40×) need OSR. Compile-dominated kernels have flat steady-state; end-to-end wall-clock is the right metric for them.

Step 3: OSR closes the one-shot-caller gap

The seven kernels with the largest Step 2 → Step 3 jump all sit in the one-shot-caller group.

Kernel	Step 2 LLVM/t2	Step 3 LLVM/t2	swing
shootout-ctype	0.37×	**0.88**×	+0.51
blind-sig	0.40×	**0.80**×	+0.40
shootout-random	0.63×	1.00×	+0.37
shootout-base64	0.78×	0.99×	+0.21
shootout-matrix	0.75×	0.95×	+0.20
shootout-ratelimit	0.79×	0.96×	+0.17
shootout-minicsv	0.79×	0.95×	+0.16

What happens at the swing. The loop's back-edge counter saturates while the loop is still running. The worker compiles a continuation. The next iteration tail-calls into LLVM-optimized code.

Compile time and end-to-end wall-clock

Compile time. How long the runtime spends compiling before steady state.
End-to-end wall-clock. How long the user actually waits.

Compile-time gain

Geometric mean: **11.1**×.
The full system finishes compilation about 11 times faster than whole-module LLVM JIT.
The **Tier-2** optimizer runs on the worker thread, off the user's critical path.

End-to-end gain

Geometric mean: **1.73**×.
The full system finishes the end-to-end workload about 73% faster on average.
The gain comes from saved compile time, even though steady-state throughput is 14% lower.

The largest wall-clock wins

The wall-clock benefit is largest on compile-dominated workloads. These workloads are the real-world targets for the architecture.

Kernel	Tier-2 time	LLVM time	speedup
`tinygo-json`	473 ms	10 361 ms	**21.9**×
`tract-onnx-image-classification`	9.7 s	136 s	**14.1**×
`spidermonkey-json`	∼9 s	∼90 s	∼10×
`spidermonkey-markdown`	∼9 s	∼90 s	∼10×
`spidermonkey-regex`	∼9 s	∼90 s	∼10×
`image-classification`	303 ms	2 667 ms	**8.8**×
`sqlite3`	1 985 ms	16 953 ms	**8.5**×

Why the wins cluster here. Each module is interpreter-shaped, reflection-heavy, host-bound, or dominated by large tensor operators. The source-to-wasm toolchain has already baked in the hot path, so LLVM cannot recover further speedup. Saving the LLVM compile cost is therefore pure win.

Where the tiered design loses ground

Long-running synthetic kernels. The **Tier-1** throughput gap is no longer offset by a meaningful cold-start saving.

Kernel	end-to-end vs. LLVM
`shootout-ctype`	0.93×
`shootout-sieve`	0.84×
`shootout-fib2`	0.90×

The central observation. Workloads split bimodally. Compile-dominated workloads win big. Long-running synthetic kernels lose a small amount. The architecture trades a modest steady-state cost for a large cold-start saving. The trade pays off most clearly on the real-world workloads the architecture targets.

Limitations and Future Work

Limitations

No SIMD lowering. Tier-1 does not lower wasm SIMD instructions. The translator rejects modules that use them. The Tier-2 promotion gate accepts only scalar signatures.

No de-tiering. Promotion is one-way. A function promoted under one workload shape stays promoted even if the shape changes.

x86-64 only. The current prototype runs on x86-64. The library backend supports aarch64, but no aarch64 port has been validated.

Conclusion

Summary

This thesis presents a **tiered compilation architecture for WasmEdge** that separates cold-start latency from peak code quality.

Three design components.

- **Tier-1**: Sea-of-Nodes baseline JIT with embedded counters and a uniform calling convention.
- **Tier-2**: mini-module synthesis drives the LLVM frontend; three ABI bridge thunks; walk-up batch policy.
- **OSR**: three-state back-edge diamond; continuation resumes a running frame mid-loop.

Headline results across 39 kernels.

- Compile time: **11.1**× faster than whole-module LLVM JIT.
- Steady-state throughput: **86%** of LLVM JIT preserved.
- End-to-end wall-clock: **1.73**× faster on average; up to 21.9× on `tinygo-json`.

Thank You

Questions?

Author: Hsiang-Yu Lee (李祥宇)
Advisor: Prof. Shih-Wei Liao
Code: WasmEdge fork with tiered JIT

Calling convention and `JitExecEnv`

Every Tier-1 function takes the same two arguments.

$$f(\texttt{env}^*,\ \texttt{args}^*) \rightarrow \texttt{void}$$

The args buffer holds one 64-bit slot per wasm parameter. One dispatch shape covers wasm-defined functions, host imports, and indirect calls.

The `JitExecEnv` struct groups state into three sets.

Group	Contents
Lookup tables	`FuncTable`, linear memory, globals, indirect-call shadow
Helper addresses	host call, memory growth, profile-saturation helpers
Promotion state	call counters, back-edge counters, `OsrEntryTable`, OSR buffer

`JitExecEnv` points to shared tables; Tier-2 publication updates `FuncTable` slots or `OsrEntryTable` slots.

Backup: scope of `JitExecEnv` writes

Field ownership keeps cross-thread synchronization simple.

- The user thread writes every field except the `OsrEntryTable` pointer.
- The worker thread writes only the `OsrEntryTable` pointer.
- Publishing a **Tier-2** batch is one atomic store into `FuncTable[i]`.
- Publishing an OSR continuation is one atomic store into `OsrEntryTable[id]`.

No locks sit on the hot dispatch path. On the supported host architecture, the dispatch load already provides the ordering the design needs.

Backup: full pseudocode of the back-edge diamond

```
slot = LOAD OsrEntryTable[loopId]
if slot != WAITING:                    // sentinel 0
  if slot == COUNTING:                 // sentinel 1
    counter = LOAD BackEdgeCounters[loopId]
    if counter < threshold:            // unsigned!
      counter += 1
      STORE BackEdgeCounters[loopId] = counter
      if counter == threshold: notify_osr(loopId)
  else:                                // READY: slot holds an entry pointer
    snapshot wasm locals into per-thread buffer
    return slot(env, locals)           // tail-call the OSR entry
fall through to the back-edge
```

Backup: OSR lineage

HotSpot pattern. A per-method back-edge counter triggers an OSR compile. The OSR compile emits a method version whose entry point is the loop header. Live state transfer uses a stack map. HotSpot has supported OSR for over two decades.

.NET 7 (2022). Added OSR with the same one-shot-caller motivation this thesis targets. The release notes describe methods called once that spend their run-time inside loops.

This thesis. Follows the HotSpot and .NET-7 lineage. Three simplifications:

- Restricted to outermost-loop back-edges.
- Live state is the wasm-level local set, not a stack map.
- Continuation is a fresh wasm function compiled through the LLVM frontend.

WebAssembly's structured control flow makes the simplifications safe.

Backup: workload categorization (39 kernels)

Frequent-call (17) blake3-scalar, bz2, gcc-loops, hashset, pulldown-cmark, quicksort, regex, shootout-gimli, shootout-heapsort, shootout-keccak, shootout-memmove, shootout-seqhash, shootout-sieve, shootout-switch, shootout-xblabla20, shootout-xchacha20, tinygo-regex

Hot-leaf (6) blind-sig, rust-compression, rust-json, rust-protobuf, shootout-ctype, shootout-ed25519

One-shot-caller (7) blind-sig, shootout-base64, shootout-ctype, shootout-matrix, shootout-minicsv, shootout-random, shootout-ratelimit

Resistant (4) richards, rust-html-rewriter, shootout-fib2, shootout-nestedloop

Compile-dominated (7) image-classification, spidermonkey-json, spidermonkey-markdown, spidermonkey-regex, sqlite3, tinygo-json, tract-onnx-image-classification

shootout-ctype and blind-sig sit in two groups. Both mechanisms must fire to close the gap.

Backup: bibliography highlights

- Click & Paleczny, *A simple graph-based intermediate representation*, IR '95. – Sea-of-Nodes.
- Click, *Global code motion / global value numbering*, SIGPLAN '95.
- Fink & Qian, *Adaptive recompilation with on-stack replacement*, CGO 2003.
- D'Elia & Demetrescu, *On-stack replacement, distilled*, TOPLAS 2018.
- Bytecode Alliance, *Wasmtime baseline compilation RFC*, 2022.
- Clark, *Firefox's streaming and tiering compiler*, Mozilla Hacks, 2018.
- Chen et al., *Accelerate RISC-V instruction set simulation by tiered JIT compilation*, VMIL 2024.
- Stogov, *dstogov/ir*: Sea-of-Nodes JIT library.